Práctica 8. Node-RED: control de un programa de Python por SSH

Orange Pi Zero 2W, Debian 12. Node-RED corre dentro de un contenedor y no ve el hardware de la placa. Para controlar `2_puntos.py` (que usa la pantalla ILI9341) desde un flujo, Node-RED entra a la placa **por SSH** y le pide a **systemd** que arranque o detenga el servicio `puntos` (práctica 7).

Requisitos:

- Docker y Compose instalados (ver `Manual_Docker_Focalboard`).
- El servicio `puntos` de la práctica 7 (`Manual_servicio_puntos`), **de usuario**.
- SSH activo en la placa: `systemctl status ssh --no-pager`.

Cambia en todo el manual `<IP-de-la-placa>` y `<nombre-de-la-placa>` por los tuyos (`hostname -I` te da la IP).

Introducción: programación low-code y Node-RED

Low-code es una forma de programar donde, en lugar de escribir todo el código, se arma el programa conectando bloques ya hechos en una interfaz gráfica y se escribe código solo donde hace falta. En Node-RED cada bloque es un **nodo** (un `inject`, un `exec`, un `debug`) y los **cables** llevan el mensaje de un nodo al siguiente: es programación **por flujos**.

Por qué importa en la industria:

- Se **prototipa más rápido**: conectar una fuente de datos con una base de datos o una alarma toma minutos, no días.
- Lo puede **entender y modificar más gente**, no solo quien programa: ingeniería de procesos, mantenimiento, operación.
- El flujo **se lee como un diagrama**, así que es más fácil de revisar y de explicar a otros.
- Sirve de **pegamento entre sistemas**: sensores, protocolos como MQTT, bases de datos, APIs web y paneles, sin reescribir cada integración.

Tabla 1. Herramientas parecidas

Herramienta	Enfoque	Comentario
Node-RED	IoT, sensores y hardware	Ligera: corre en una Orange Pi. Es la que usamos
n8n	Automatizar servicios web (correo, hojas de cálculo, APIs)	Muy famosa, pero **pesada** para una placa pequeña
Apache NiFi	Flujos de datos a gran escala	De empresa, requiere un servidor
Make y **Zapier**	Automatización entre servicios web	En la nube, de pago

Por qué Node-RED aquí. n8n y NiFi nacieron para servidores y servicios web, y piden más memoria y procesador de lo que tiene una Orange Pi Zero 2W. Node-RED nació para el IoT, trae nodos para MQTT, serial y GPIO, y corre bien en equipos pequeños. Por eso es el programa adecuado para este curso. Fuente: https://nodered.org/

Objetivo

Instalar Node-RED con Docker en la Orange Pi Zero 2W y, desde un flujo de Node-RED, **arrancar y detener un programa de Python** que corre en la placa. Node-RED lo controla **a través de SSH**, pidiéndole a systemd que inicie o pare el servicio. El programa que se usa para comprobar el funcionamiento es `2_puntos.py`, el de los puntos rebotando en la pantalla ILI9341.

Ejercicio 1. Instalar Node-RED con Docker

Fuente oficial: https://nodered.org/docs/getting-started/docker

1.1 Carpeta persistente

Todo lo que Node-RED guarda (flujos, llaves SSH) vive aquí y sobrevive al borrar el contenedor.

```
mkdir -p ~/Documents/nodered_pers/data
cd ~/Documents/nodered_pers
```

Node-RED corre dentro del contenedor con el usuario de uid 1000, el mismo que `orangepi`. Si Docker se queja de permisos:

```
sudo chown -R 1000:1000 ~/Documents/nodered_pers/data
```

1.2 Crear y correr el contenedor

Debes estar dentro de la carpeta `nodered_pers`: el comando usa `$PWD`, la carpeta actual, para encontrar `data`. Desde otra carpeta, Docker montaría un `data` equivocado.

```
cd ~/Documents/nodered_pers
docker run -d \
  -p 1880:1880 \
  -v "$PWD/data":/data \
  --name nodered \
  --restart=no \
  nodered/node-red
```

- `-d`: segundo plano. `-p 1880:1880`: puerto `placa:contenedor`.
- `-v`: la carpeta `data` de la placa (`$PWD/data`, o sea `~/Documents/nodered_pers/data`) es el `/data` del contenedor.
- `--restart=no`: no arranca con la placa; se inicia con `docker start nodered`.

1.3 Probar

```
docker ps
```

Debe aparecer `nodered` con estado `Up`. En el navegador de tu PC:

```
http://<nombre-de-la-placa>:1880
```

Manejo básico: `docker stop nodered`, `docker start nodered`, `docker logs nodered`.

Ejercicio 2. Dejar listo el servicio `puntos`

Node-RED solo va a arrancarlo y detenerlo. Que no arranque con la placa:

```
systemctl --user disable puntos
systemctl --user stop puntos
systemctl --user status puntos --no-pager
```

Debe decir `inactive (dead)`. Prueba a mano que el servicio funciona antes de involucrar a Node-RED:

```
systemctl --user start puntos     # se ven los puntos en la pantalla
systemctl --user stop puntos      # se detienen
```

Si `start` falla con `Device or resource busy`, hay otro programa usando la pantalla: `pgrep -af .py` y `pkill -f <programa>`.

Ejercicio 3. SSH sin contraseña desde el contenedor

Node-RED no puede teclear contraseñas. Se usa un par de llaves: la **privada** se queda dentro del contenedor y la **pública** se autoriza en la placa. Quien tenga la privada puede entrar a la placa: no se comparte.

3.1 Generar la llave dentro del contenedor

Aquí se modifica el contenedor, no el usuario de la placa: la llave se crea en el `/data/.ssh` del contenedor. Como `/data` es la carpeta `nodered_pers/data` montada, el mismo archivo se ve en la placa en `~/Documents/nodered_pers/data/.ssh`, pero `~/.ssh` (el de `orangepi`) no cambia.

Qué hace `docker exec`. Se escribe `docker exec <contenedor> <comando>`: el comando **se ejecuta dentro del contenedor** `nodered`, con su propio sistema de archivos, y no en el sistema operativo de la placa. Por eso `mkdir -p /data/.ssh` crea la carpeta en el `/data` del contenedor, no en el `/data` de la placa. Lo que escribes sin `docker exec` se ejecuta en la placa.

Tabla 2. Dónde se ejecuta cada comando

Comando	Dónde se ejecuta
`docker exec nodered mkdir -p /data/.ssh`	Dentro del contenedor
`mkdir -p ~/.ssh`	En la placa

La llave se guarda en `/data/.ssh`, la carpeta persistente. **No hace falta estar en `nodered_pers`**: `docker exec` funciona desde cualquier carpeta, porque el contenedor ya tiene la carpeta montada.

```
docker exec nodered mkdir -p /data/.ssh
docker exec nodered chmod 700 /data/.ssh
docker exec nodered ssh-keygen -t ed25519 -N "" -f
/data/.ssh/id_ed25519
```

3.2 Autorizar la llave pública en la placa

Aquí se modifica la placa, no el contenedor: se edita el `~/.ssh` del usuario `orangepi`, que es el que consulta el servidor SSH al recibir una conexión. Estos comandos se escriben en la terminal de la placa, no con `docker exec` (salvo el `cat` de la llave):

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
docker exec nodered cat /data/.ssh/id_ed25519.pub >>
~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

La línea agregada termina en `node-red@<id-del-contenedor>`: esa es la que se borra para revocar el acceso.

3.3 Probar la conexión desde el contenedor

```
docker exec nodered ssh \
  -i /data/.ssh/id_ed25519 \
  -o StrictHostKeyChecking=no \
  orangepi@<IP-de-la-placa> \
  echo hola
```

Es **un solo comando** en varias líneas: cada `\` al final continúa en la siguiente, y la última línea (`echo hola`) es lo que se ejecuta en la placa. Si se copia por partes, SSH imprime su ayuda (`usage: ssh ...`) porque le falta el destino. Debe imprimir `hola` sin pedir contraseña. `-i` indica la llave privada y `StrictHostKeyChecking=no` acepta la huella de la placa sin preguntar.

3.4 Probar systemctl por SSH

```
docker exec nodered ssh \
  -i /data/.ssh/id_ed25519 \
  -o StrictHostKeyChecking=no \
  orangepi@<IP-de-la-placa> \
  systemctl --user is-active puntos
```

Debe imprimir `inactive`.

Si dice `Failed to connect to bus`, a la sesión le falta la variable del administrador de usuario. Se agrega delante de `systemctl`:

```
... orangepi@<IP-de-la-placa> XDG_RUNTIME_DIR=/run/user/1000 systemctl
--user is-active puntos
```

Si aun así falla, probar `sudo loginctl enable-linger orangepi` en la placa.

Ejercicio 4. El flujo en Node-RED

Abre `http://<nombre-de-la-placa>:1880`. Un solo nodo `exec` hace las tres acciones: cada `inject` le manda en `msg.payload` lo que va al final del comando. El flujo completo se ve así:

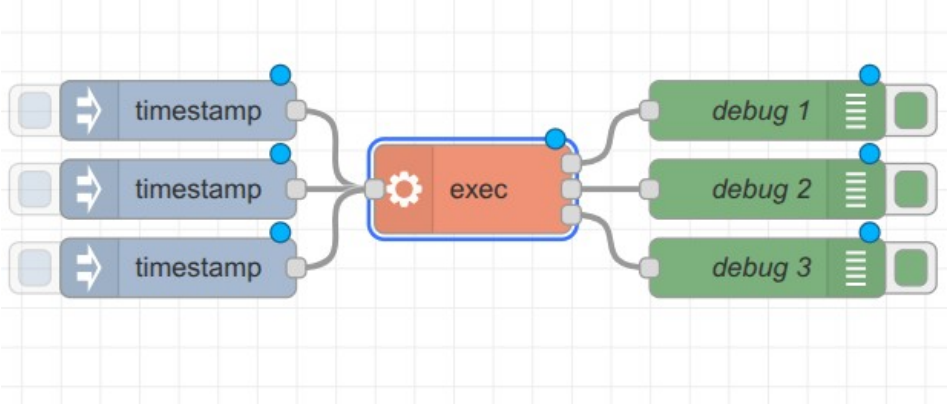


Figura 1. Flujo: tres inject conectados a un exec, cuyas tres salidas van a tres debug

4.1 Nodos

1. Tres nodos `inject`, cada uno con `msg.payload` de tipo **string**:

Tabla 3. Los tres inject

Nombre del inject	Payload
Arrancar	`start puntos`
Detener	`stop puntos`
Estado	`is-active puntos`

Así se ve el `inject` **Arrancar**: `msg.payload` de tipo texto (`az`) con el valor `start puntos`.

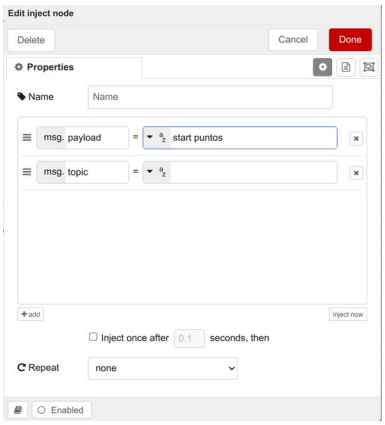


Figura 2. Configuración del inject Arrancar

2. Un nodo `exec` (sección advanced).
3. Tres nodos `debug` para sus tres salidas: stdout, stderr y código de retorno.
4. Conectar los tres `inject` → `exec`, y las tres salidas del `exec` → un `debug` cada una.

4.2 Configurar el `exec`

En **Command** (una sola línea):

```
ssh -q -i /data/.ssh/id_ed25519 -o StrictHostKeyChecking=no
orangepi@<IP-de-la-placa> systemctl --user
```

- Marcar **Append msg.payload**: el `inject` agrega `start puntos`, `stop puntos` o `is-active puntos`.
- **No** marcar `spawn()`: `systemctl start` termina al instante, ya no hay un programa que se quede corriendo.
- `-q` silencia las advertencias de SSH.

Así debe verse la configuración, con Append marcado y la salida en modo exec (when the command is complete):

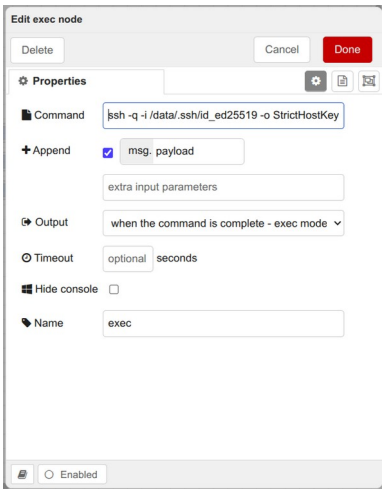


Figura 3. Configuración del nodo exec

4.3 Desplegar y probar

Antes de empezar, abre el panel de mensajes con la pestaña **debug** (el icono del insecto, marcado en rojo en la imagen): ahí aparecen las salidas de los tres `debug`. Si el panel no está abierto, no verás nada aunque el flujo funcione.

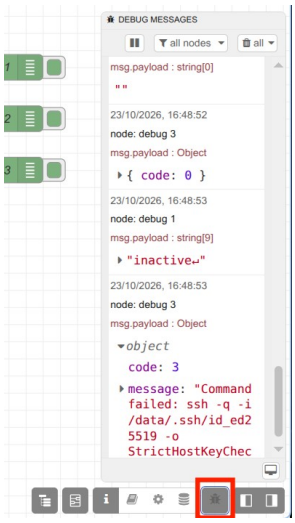


Figura 4. Panel debug de Node-RED, con el botón del insecto resaltado

1. **Deploy**.
2. **Estado**: en debug 1 (stdout) debe salir `"inactive\n"`. En debug 3 aparece `{ code: 3, message: "Command failed: ssh -q -i ..." }`: **es normal**. `systemctl is-active` devuelve 3 cuando el servicio no está activo, y Node-RED interpreta cualquier código distinto de 0 como "falló".
3. **Arrancar**: los puntos aparecen en la pantalla. En debug 1 sale una cadena vacía (`""`, `systemctl start` no imprime nada) y en debug 3 `{ code: 0 }`. Debug 2 (stderr) no debe mostrar nada.
4. **Estado**: ahora `active`, código `0`.
5. **Detener**: los puntos se detienen. La pantalla conserva el último cuadro: el programa no la borra al terminar.

Confirmar desde la terminal de la placa:

```
systemctl --user status puntos --no-pager
```

Si algo falla

Tabla 4. Mensajes de error comunes

Mensaje	Causa probable	Qué hacer
`Connection refused`	`sshd` no está activo en la placa	`sudo systemctl enable --now ssh`
`Permission denied (publickey)`	La llave no está en `authorized_keys`, o los permisos están abiertos	Repetir 3.2; `chmod 700 ~/.ssh` y `chmod 600 ~/.ssh/authorized_keys`
`Failed to connect to bus`	La sesión SSH no tiene la variable del administrador de usuario	Ver la nota al final de 3.4
`Device or resource busy`	Otro programa usa la pantalla	`pgrep -af .py` y `pkill -f <programa>`
`Unit puntos.service not found`	El servicio no existe	Repetir `Manual_servicio_puntos` (de usuario)
`ssh: not found` en el `exec`	La imagen de Node-RED no trae el cliente SSH	Avisar al profesor

Deshacer (opcional, solo en caso de falla)

Quitar el acceso de Node-RED a la placa: en la placa, borrar de `~/.ssh/authorized_keys` la línea que termina en `node-red@<id-del-contenedor>`:

```
sed -i '/node-red@/d' ~/.ssh/authorized_keys
```

Borrar la llave del contenedor, y el contenedor:

```
docker exec nodered rm -rf /data/.ssh
docker stop nodered
docker rm nodered
```

Los flujos siguen en `~/Documents/nodered_pers/data`.

Fuentes:

- Node-RED con Docker: https://nodered.org/docs/getting-started/docker
- `man systemctl`, `man ssh`
- Manuales del curso: `Manual_servicio_puntos` (práctica 7), `Manual_Docker_Focalboard`

Reporte

Elabora un reporte siguiendo el formato de la plantilla Reporte_plantilla.docx proporcionada por el profesor. Incluye capturas de tu flujo en Node-RED, del panel debug con la salida de **Arrancar**, **Estado** y **Detener**, y de la pantalla con los puntos en movimiento. Los siguientes puntos te pueden ayudar a redactar tu reporte.

1. Objetivos. Escribe los objetivos que identificaste en la práctica. Para ayudarte, puedes identificar el tema abordado y el aprendizaje obtenido.
2. Resultados y análisis. Presenta los resultados que obtuviste en cada parte de la práctica y analízalos.
3. Conclusiones. Puedes redactar tus conclusiones basado en las siguientes preguntas:
   a. Compara el control del programa con Node-RED contra hacerlo de forma tradicional (por ejemplo, un script en Python o Bash que arranque y detenga el servicio). ¿Qué fue más rápido de armar y más claro de leer en cada caso? ¿Qué te costó más?
   b. Aunque el flujo es visual, hubo que configurar en texto Docker, SSH y systemd. ¿Qué te dice eso sobre los alcances y los límites de la programación low-code?
   c. ¿En qué tipo de proyecto usarías Node-RED y en cuál preferirías programación tradicional? Justifica con criterios como mantenimiento, rendimiento, control de versiones y quién más trabajará en el sistema.